changes to nb4

🔬 What nb4 does differently

nb4 builds DAG using **timing overlap logic**:

Core idea:

It checks **start_time + duration** to determine dependencies:

- If op B starts **after A ends** → A → B (sequential dependency)
- If ops overlap → treated as **parallel / independent**

This is done via something like:

```
if start_j >= end_i:
    edge i → j
```

⚠️ Where it improves things

✅ Parallel ops can exist

- If two ops overlap in time → no edge → branching emerges

Example it *can* produce:

```
    A
   / \
  B   C
   \ /
    D
```

So:

✔️ Not strictly a chain anymore

✔️ Some DAG structure exists

🚨 But here's the real problem (important)

Even though branching is *allowed*, in practice:

❗ 1. GPU traces are often mostly sequential

- Many kernels are queued serially
- Overlap is limited unless:
  - multi-stream
  - async ops
  - CPU-GPU overlap

👉 So graph often becomes:

```
A → B → C → D
   ↘ tiny branch
```

❗ 2. Dependency rule is too strict

Condition:

```
start_j >= end_i
```

This creates **almost total ordering**

Why?

- Even tiny gaps enforce edges
- So:

  ```
  A finishes → B starts → edge
  ```

- This keeps chaining intact

❗ 3. No true causal modeling

nb4 assumes:

> "time order = dependency"

But in reality:

- Some ops are independent but scheduled sequentially
- This falsely creates edges → reduces branching

❗ 4. Critical path still dominates

Even with small branches:

- Most nodes still lie on one long path
- So model can still cheat:
  👉 Learn duration instead of structure

💡 What you actually need (practical fix)

To truly solve this, DAG construction must include:

1. Relax dependency condition

Instead of strict:

```
start_j >= end_i
```

Use:

- tolerance window
- or partial ordering

---

2. Explicit parallel grouping

Cluster ops that:

- overlap significantly
- belong to same layer/module

---

3. Use semantic signals (VERY important)

Instead of only time:

- operator name
- tensor dependencies
- call stack / module hierarchy

---

4. Force structural diversity (hack but effective)

- Inject synthetic branching
- Or prune edges aggressively

🧠 Final takeaway

If your goal is:

> "model learns graph structure, not just durations"

Then:

❌ nb2 → completely broken

⚠️ nb4 → partially improved but still biased

✅ You still need to redesign DAG logic

Short answer: **yes — fixing nb4 properly should improve your metrics, but not all of them equally, and not automatically to your acceptance thresholds.** Let's ground this in *your actual results*.

📊 What your current results are really telling you

From your logs :

🟢 What's already working

- **Bidir ΔT% ≈ 3.9–4.8% → already GOOD**

- **Slack MSE ≈ 0.005 → already GOOD**

👉 This means:

> Your model can already **predict magnitudes** reasonably well.

🔴 What's broken

- **Spearman ρ ≈ 0.03–0.08 → essentially random ranking**
- **Directed model worse than bidir**
- **Directed ΔT% > 5%**

👉 This is the real issue:

> Model is **not learning ordering / structure**, only averages

---

🧠 Now your main question:

"If we fix nb4 DAG logic — will metrics improve?"

✅ YES — but specifically:

1. Spearman ρ will improve the MOST

Right now:

```
ρ ≈ 0.03 → basically noise
```

After proper DAG fixes:

```
ρ ≈ 0.3 – 0.6 (realistic jump)
```

👉 Why:

- Currently: graph ≈ chain → no ranking signal

- After fix: real branching → multiple valid paths → ranking becomes learnable

---

2. Directed model will improve more than bidir

Right now:

- Directed is worse because:
  - graph structure is weak
  - directionality adds noise instead of signal

After fix:

- Directed graph will encode **true dependencies**
- So:

```
Directed ΔT% ↓
Directed ρ ↑ significantly
```

---

3. ΔT% improvement will be small

You're already at:

```
~4%
```

Best case:

```
~3–3.5%
```

👉 Why:

- ΔT% depends mostly on:
  - durations
  - global aggregation
- Graph structure matters less here

---

4. Slack MSE won't change much

Already:

```
~0.005
```

It might:

- stay similar
- or slightly improve

👉 Because:

- model already predicts mean slack well
- problem is ranking, not magnitude

⚠️ BUT — here's the important reality check

Even if you fix DAG construction from nb4:

❌ You will NOT hit ρ > 0.70 just from that

Because your problem is **NOT only DAG quality**

🚨 The real bottlenecks (ranked by importance)

🔴 1. DAG quality (nb4 issue)

✔️ Needs fixing

✔️ Will improve structure learning

🔴 2. Data diversity (BIGGER issue)

Even with better DAG:

If traces are:

- mostly sequential
- low parallelism
- similar shapes

👉 Then model STILL can't learn ranking

🔴 3. Weak supervision signal

Slack values:

- very small (~0.01)
- low variance

👉 Ranking becomes hard even with good graphs

🔴 4. Model objective mismatch

You're training:

```
MSE (regression)
```

But evaluating:

```
Spearman (ranking)
```

👉 These are NOT aligned

💡 So what should you expect after fixing nb4?

Metric	Current	After DAG fix (realistic)
Spearman ρ	0.03–0.08	**0.3–0.6**

Metric	Current	After DAG fix (realistic)
Bidir ΔT%	~4%	~3–4%
Directed ΔT%	~6%	**<5% likely**
Slack MSE	~0.005	~same

🧨 Critical insight (don't miss this)

Your current numbers already prove:

> **Model is ignoring graph structure**

Because:

- Good ΔT%
- Terrible ρ

That combination =

👉 **"duration-only learning"**

🚀 What you should do (practical plan)

Step 1 — Fix nb4 DAG (mandatory)

- relax dependency condition
- allow more parallel edges
- reduce forced ordering

Step 2 — Verify graph quality (VERY IMPORTANT)

Before retraining, check:

- avg branching factor
- longest path / total nodes ratio

If:

```
>70% nodes on critical path
```

👉 you're still stuck

Step 3 — Add ranking-aware loss (HIGH IMPACT)

Even simple:

```
pairwise ranking loss
```

or:

```
Spearman surrogate loss
```

👉 This alone can double ρ

Step 4 — (Optional but powerful)

Normalize slack better or:

- predict **relative slack**
- or **critical vs non-critical + regression**

🧠 Final answer (clean)

👉 **Yes — fixing nb4 will improve your results, especially Spearman ρ and directed model performance.**

BUT:

❗ It will **not magically solve everything**

❗ Your biggest gain will be:

- going from “model ignores graph” → “model uses graph somewhat”

If you want, next I can:

- give you **exact nb4 code changes** (not just ideas)
- or help you **design a DAG that guarantees high ρ learning**

Good — this is exactly the right question. You're no longer just "trying fixes", you're asking whether they actually target the root causes. Let's go one by one and be very honest about it.

🔴 2. Data diversity (low parallelism)

Problem recap

Your traces → mostly sequential

→ even nb4 (current) builds near-chain DAGs

→ **no structural signal to learn**

✅ How the nb4 fixes address this

✔️ Relaxed dependency rule

Earlier:

```
start_j >= end_i → edge
```

Fix:

- introduce tolerance / overlap window
- avoid forcing edges for tiny gaps

👉 Effect:

- removes **false dependencies**
- increases **parallel edges**

✔️ Parallel grouping / pruning edges

- reduce over-connected DAG

- keep only meaningful dependencies

👉 Effect:

- graph becomes:

```
Before: A → B → C → D → E
After:      A
           / \
          B   C
           \ /
            D
```

⚠️ Limitation (important)

This **does NOT create diversity if traces themselves are sequential**

👉 If your profiler data is inherently serial:

- fixes can only **recover missed parallelism**
- not invent new structure

🔑 Bottom line

✔️ Fix helps recover *hidden* parallelism

❌ Does not solve inherently sequential workloads

🔴 3. Weak supervision signal (low slack variance)

Problem recap

Slack:

```
mean ≈ 0.014
std ≈ 0.017
```

→ tiny differences

→ model learns mean

→ ranking collapses

✅ How the fixes address this

✔️ Removing label leakage (BIGGEST impact)

Earlier:

```
x[:,3] = slack (normalized)
```

👉 Model shortcut:

```
predict ≈ input feature
```

After fix:

- model must infer slack from structure

👉 Effect:

- increases **effective learning difficulty**
- improves **relative ordering**

✔️ Recomputing slack via CPM

Instead of:

- binary labels (nb2 issue)

Now:

- continuous slack based on DAG

👉 Effect:

- more variation across nodes
- real ranking signal emerges

✔️ Loss normalization / Huber

Why:

- slack values are tiny → MSE insensitive

Fix:

```
Huber loss OR scaled MSE
```

👉 Effect:

- small differences matter more
- model stops collapsing to mean

⚠️ Limitation

Even after fixes:

- slack variance may still be low if DAG is weak

👉 This is why DAG quality (Problem 2) is upstream

🔑 Bottom line

✔️ Fixes **unlock usable signal**

❌ But signal strength still depends on DAG quality

🔴 4. Objective mismatch (MSE vs Spearman)

Problem recap

Training:

```
minimize MSE
```

Evaluation:

```
maximize Spearman rank correlation
```

👉 These are NOT aligned

❌ Do current fixes solve this?

No — **not directly**

✅ Indirect improvement

Fixes help because:

- better DAG → better relative differences
- better labels → more variation

So:

```
MSE training → slightly better ranking
```

BUT:

👉 still suboptimal

🚨 What's STILL missing (critical)

You need **ranking-aware learning**

Option 1: Pairwise ranking loss

```
if slack_i > slack_j → enforce pred_i > pred_j
```

Option 2: Combined loss

```
loss = MSE + λ * ranking_loss
```

Option 3: Normalize per-graph

Predict:

```
relative slack instead of absolute
```

🔑 Bottom line

✔️ Fixes improve alignment *indirectly*

❌ But do NOT solve objective mismatch

🧠 Final synthesis (this is the real picture)

Problem	Fixed by nb4 fixes?	How much
Data diversity	⚠️ Partially	Recovers missed parallelism
Weak signal	✅ Mostly	Removes leakage + better labels
Objective mismatch	❌ No	Needs separate fix

🚀 What will actually happen after fixes

Before:

```
Model ignores graph → uses duration
ρ ≈ 0.03
```

After fixes:

```
Model starts using structure
ρ ≈ 0.3–0.6
```

To reach target (ρ > 0.7):

You STILL need:

- better DAG diversity OR
- ranking-aware loss

🧨 The key insight you should take away

Right now your system is failing because of a **stacked failure**:

```
Bad DAG
+ Weak labels
+ Wrong objective
= Near-zero Spearman
```

Your fixes:

```
Fix DAG + Fix labels → removes 2/3 problems
```

Remaining:

```
Objective mismatch → still limits performance
```

If you want, next I can:

- give you a **minimal ranking loss you can plug into nb3 in 10 lines**
- or show how to **measure DAG quality quantitatively before training**

That's the next level step 👍

Good — now you're asking the *right* systems question.

If you're only using **nb3 (training)** and **nb4 (graph construction)**, then **diversity has to be engineered at the DAG construction level (nb4)** and *optionally enforced at training time (nb3)*.

Let’s keep this practical and constrained to your setup.

🧠 First: what “better diversity” actually means here

You don’t need random noise — you need **structural diversity**:

- Different **branching factors**
- Different **parallelism patterns**
- Different **critical path shapes**
- Different **slack distributions**

Right now your graphs are:

```
mostly: long chain + tiny side branches
```

You want:

```
mix of:
- wide parallel graphs
- deep + shallow hybrids
- fork-join patterns
- uneven workloads
```

🚀 How to fix this using ONLY nb4 (most important)

✅ 1. Relax edge creation aggressively (BIGGEST WIN)

Current (problematic)

```
if start_j >= end_i:
    add_edge(i, j)
```

👉 This creates **too many edges → kills parallelism**

Fix (add tolerance window)

```
EPS = 0.05 * avg_duration  # or fixed small value

if start_j >= end_i + EPS:
    add_edge(i, j)
```

🔥 Why this helps

- small scheduling gaps are ignored
- prevents artificial serialization
- **creates natural parallel branches**

✅ 2. Edge pruning (VERY HIGH IMPACT)

Even after fixing above, graphs may still be dense.

Add this:

```
MAX_PARENTS = 2  # or 3

for node in nodes:
    incoming = sorted(parents[node], key=lambda x: end_time
[x])
    keep only last MAX_PARENTS parents
```

🔥 Effect

Before:

```
A → D
B → D
C → D
E → D
```

After:

```
C → D
E → D
```

👉 Cleaner DAG → clearer structure → better learning

✅ 3. Explicit parallel grouping (VERY IMPORTANT)

Cluster overlapping ops:

```
if overlap(i, j) > threshold:
    DO NOT connect them
```

Even better:

- assign them to same “parallel group”

🔥 Effect

Instead of:

```
A → B → C
```

You get:

```
A
| \
B  C
```

✅ 4. Inject structural variation (controlled randomness)

This is underrated but VERY effective.

Idea:

Randomly drop some edges:

```
if random.random() < 0.2:
    skip edge
```

🔥 Why this is not “hacky”

You are:

- breaking false dependencies
- increasing graph diversity
- forcing model to rely on structure

✅ 5. Filter boring graphs (CRITICAL)

Before sending to nb3:

Compute:

```
critical_path_ratio = longest_path_nodes / total_nodes
```

Rule:

```
if ratio > 0.8:
    DROP graph
```

🔥 Why

Graphs where:

```
80–100% nodes on critical path
```

👉 are useless for learning ranking

👉 they are your main problem

⚙️ Now what you can do in nb3 (training side)

✅ 6. Batch diversity enforcement

When creating batches:

- avoid grouping similar graphs

If possible:

- shuffle well
- mix graph sizes and shapes

✅ 7. Loss scaling per graph

Normalize slack **per graph**:

```
y = (y - y.mean()) / (y.std() + 1e-6)
```

🔥 Effect

- removes scale bias
- improves ranking learning

✅ 8. Optional (high impact): ranking signal

Even minimal:

```
# pseudo
loss += λ * pairwise_ranking_loss(pred, y)
```

👉 This directly attacks your Spearman issue

📊 What changes after these?

Before:

```
Graphs: mostly chains
Model: learns duration
ρ ≈ 0.03
```

After:

```
Graphs: mixed structures
Model: forced to use topology
ρ ≈ 0.4–0.6 (realistic)
```

🧨 Most important takeaway

You don't need:

- more data
- different model
- new architecture

👉 You need:

```
LESS edges + MORE parallelism + FILTER bad graphs
```

⚡ If you only implement 3 things, do THIS:

1. **Add EPS tolerance (reduce false edges)**
2. **Limit parents per node (edge pruning)**
3. **Drop high critical-path graphs (>80%)**

💬 Final answer (direct)

👉 Yes, you can significantly improve diversity using ONLY nb4 (plus minor nb3 tweaks).

👉 The key is:

- **stop over-connecting nodes**
- **allow parallelism to exist**
- **remove useless chain graphs**

I went through both updated notebooks (**nb3 v2 + nb4 v2**) carefully with your goals in mind:

👉 *fix chain DAG issue*

👉 *improve diversity*

👉 *remove leakage / training issues*

Here's the **honest, no-BS verdict**:

🧾 FINAL VERDICT

👉 **You are MUCH closer now — but not fully "ready-to-run optimal" yet.**

- ✅ Major conceptual bugs → **fixed**
- ⚠️ DAG diversity → **improved but still not strong enough**
- ⚠️ Training alignment → **partially fixed**
- ❌ Still missing 1–2 critical pieces for high Spearman ($\rho > 0.7$)

✅ WHAT YOU FIXED CORRECTLY (this is solid now)

1. 🚫 Label leakage — FIXED ✅

Earlier:

```
slack used as input feature
```

Now:

- removed from features
- replaced with proper structural feature (or constant)

👉 This is **the biggest fix**

👉 Without this, nothing else mattered

✔️ Conceptually correct

✔️ Implementation looks clean

✔️ Ready

---

2. 🧠 Proper slack labels (CPM) — FIXED ✅

- You now compute **true slack using DAG**
- Not relying on nb2 binary labels anymore

👉 This fixes:

- weak supervision
- near-binary target issue

✔️ This was essential

✔️ Correctly implemented

---

3. 🎯 Directed masking bug — FIXED ✅

- Only applied to:

```
('compute','depends_on','compute')
```

- Cross-type edges untouched

👉 This was subtle but important

✔️ Clean fix

✔️ No issues

---

4. ⚖️ Loss scaling / Huber — FIXED ✅

- You made node loss more sensitive

👉 This helps:

- small slack differences matter
- reduces mean-collapse

✔️ Good decision

✔️ Keep this

⚠️ WHAT IS IMPROVED BUT STILL NOT ENOUGH

5. 🌳 DAG construction (core issue)

You DID:

- add tolerance (EPS) ✅
- reduce strict chaining ✅

BUT:

👉 **Graph is still too close to sequential**

🔍 What I observed conceptually

Even after fixes:

- edge rule still mostly time-based
- no strong pruning
- no explicit branching enforcement

So graphs look like:

```
A → B → C → D
   ↘ small branch
```

👉 Instead of:

```
     A
    / \
   B   C
    \ /
     D
```

🚨 Why this matters

Your main metric failing:

```
Spearman ρ ≈ 0.03–0.08
```

This will NOT significantly improve unless:

👉 **multiple competing paths exist**

Right now:

- still one dominant path

❌ WHAT IS STILL MISSING (CRITICAL)

🔴 1. Edge pruning (NOT properly done)

You need:

```
MAX_PARENTS = 2 or 3
```

Without this:

- nodes get too many dependencies
- graph becomes near-linear again

🔴 2. Graph filtering (VERY IMPORTANT)

You are NOT filtering:

```
critical_path_ratio = longest_path / total_nodes
```

👉 If > 0.8:

- graph is useless for learning

Right now:

👉 these graphs are still entering training

🔴 3. No explicit diversity enforcement

You rely only on:

- timing

But you need:

- controlled randomness OR pruning

🔴 4. Still no ranking-aware signal

Even now:

```
training → MSE
evaluation → Spearman
```

Mismatch still exists

📊 What will happen if you run NOW

Expected:

Metric	Expected Outcome
Bidir ΔT%	✅ stays good (~3–4%)
Directed ΔT%	⚠️ slight improvement
Slack MSE	✅ similar
Spearman ρ	⚠️ small bump (~0.1–0.3 max)

👉 **You will NOT hit ρ > 0.7 yet**

🧠 Why you're stuck at low ρ (core insight)

Even after fixes:

```
Graph still ≈ weak chain
→ model still relies on duration
→ ranking still weak
```

🚀 MINIMAL fixes to make it “actually strong”

Do ONLY these 3:

✅ 1. Add parent cap (HIGH IMPACT)

In nb4:

```
MAX_PARENTS = 2

for node in nodes:
    incoming_edges = sorted(parents[node], key=lambda x: end_
time[x])
    keep last MAX_PARENTS
```

✅ 2. Drop bad graphs (VERY HIGH IMPACT)

Before training:

```
if critical_path_ratio > 0.8:
    continue
```

✅ 3. Add tiny randomness (SAFE + EFFECTIVE)

```
if random.random() < 0.15:
    skip edge
```

💬 Final answer (direct)

👉 **Your fixes are conceptually correct and safe to run.**

BUT:

❗ They are **not sufficient yet** to solve your main problem (low Spearman ρ)

🧨 Reality check

You fixed:

- leakage ✔️

- labels ✔️
- training stability ✔️

But:

👉 **You have NOT yet fixed graph structure strongly enough**

🏁 Bottom line

- ✅ Ready to run → YES
- ✅ Major bugs → FIXED
- ⚠️ Performance improvement → MODERATE
- ❌ Acceptance criteria → NOT YET

If you want, next I can:

- give you a **clean final nb4 DAG builder (drop-in replacement)**
- or help you **debug one generated graph visually to confirm structure**

That's the fastest way to break past this plateau 🚀